EE 475 Introduction to Image Processing
Homework 01 Report

Melih Kayhan Pala
Department of Electrical and Electronics Engineering
Boğaziçi University

October 14, 2026

1 Scaling

1.1 Theoretical Formulation

Let $A \in \mathbb{R}^{M \times N}$ denote a two-dimensional grayscale image matrix with spatial resolution $M \times N$. To linearly map the pixel intensity values of A into the normalized range $[0, 1]$, we apply the min-max normalization transformation $f : \mathbb{R}^{M \times N} \to [0, 1]^{M \times N}$ defined element-wise as:

$$A_{\text{scaled}}(i, j) = \frac{A(i, j) - A_{\min}}{A_{\max} - A_{\min}}, \quad \forall i \in \{0, \ldots, M-1\},\ j \in \{0, \ldots, N-1\} \tag{1}$$

where the global minimum and maximum intensities of the matrix are defined as:

$$A_{\min} = \min_{i,j} A(i, j) \quad \text{and} \quad A_{\max} = \max_{i,j} A(i, j) \tag{2}$$

In the degenerate case of a uniform-intensity image where $A_{\max} = A_{\min}$, the output matrix is set to $A_{\text{scaled}}(i, j) = 0$ to avoid division by zero.

Required Functions for Normalization:

- **Type Casting Function (`astype(float64)`):** Converts integer pixel representations (e.g., `uint8`) to floating-point format to prevent truncation during division.
- **Global Minimum Function (`min`):** Extracts the lowest pixel intensity $A_{\min}$ to shift the baseline intensity to zero.
- **Global Maximum Function (`max`):** Extracts the highest pixel intensity $A_{\max}$ to compute the dynamic range $(A_{\max} - A_{\min})$.
- **Element-wise Arithmetic Operators:** Array subtraction and division to map intensities into $[0, 1]$.

1.2 Implementation

The Python implementation of the scaling routine is provided in Appendix A (`scale_image`).

2 Interpolation

2.1 Resolutions of Provided Image Files

The spatial dimensions (Height × Width) of the original chronometer image and its sub-sampled versions are presented in Table 1. Note that `Chronometerlow5.tif` (428×428) corresponds to $\lceil 2136/5 \rceil \times (2140/5)$.

Table 1: Spatial resolutions and sub-sampling factors of the provided chronometer images.

File Name	Height (px)	Width (px)	Sub-sampling Factor ($S_y \times S_x$)
`Chronometer.tif`	2136	2140	1×1 (Original)
`Chronometerlow2.tif`	1068	1070	2×2
`Chronometerlow34.tif`	712	535	3×4
`Chronometerlow5.tif`	428	428	5×5

2.2 Bilinear and Bicubic Interpolation Implementation

Both interpolation routines map target pixel coordinates (i, j) in the 2136×2140 grid back to continuous coordinates (x, y) in the reduced-resolution grid:

- **Bilinear Interpolation:** Uses a 2×2 neighborhood and computes a linear combination weighted by fractional distances Δx and Δy.
- **Bicubic Interpolation:** Uses a 4×4 neighborhood evaluated via separable 1D convolutions with the Keys cubic kernel ($a = -0.5$).

The complete implementations are provided in Appendix A.

2.3 Performance Evaluation and Discussion

To quantitatively evaluate reconstruction accuracy between the normalized original image I and interpolated image $\hat{I}$, we compute Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR):

$$\text{MSE} = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \Big(I(i,j) - \hat{I}(i,j) \Big)^2, \qquad \text{PSNR} = 10 \log_{10} \left(\frac{1.0^2}{\text{MSE}} \right) \tag{3}$$

Performance Report (100 Words):
Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) were evaluated between the normalized original image and interpolated reconstructions (Table 2). As sub-sampling increases from factor 2 to 5, PSNR drops from $\sim$ 30.8 dB to $\sim$ 24.1 dB due to severe aliasing and high-frequency loss. Visually (Figure 1), bicubic interpolation produces noticeably sharper edges on dial numerals and tick marks than bilinear interpolation. However, bilinear interpolation achieves slightly lower MSE (higher PSNR) because its smoothing nature suppresses background grain and avoids the edge ringing (overshoot) artifacts introduced by negative side-lobes of the bicubic convolution kernel across high-contrast boundaries.

Table 2: Quantitative comparison of Bilinear and Bicubic interpolation on $[0, 1]$ scaled images.

	Bilinear Interpolation		Bicubic Interpolation	
Input Image	**MSE**	**PSNR (dB)**	**MSE**	**PSNR (dB)**
`Chronometerlow2.tif`	0.000821	30.86	0.000832	30.80
`Chronometerlow34.tif`	0.003302	24.81	0.003548	24.50
`Chronometerlow5.tif`	0.003797	24.21	0.003939	24.05

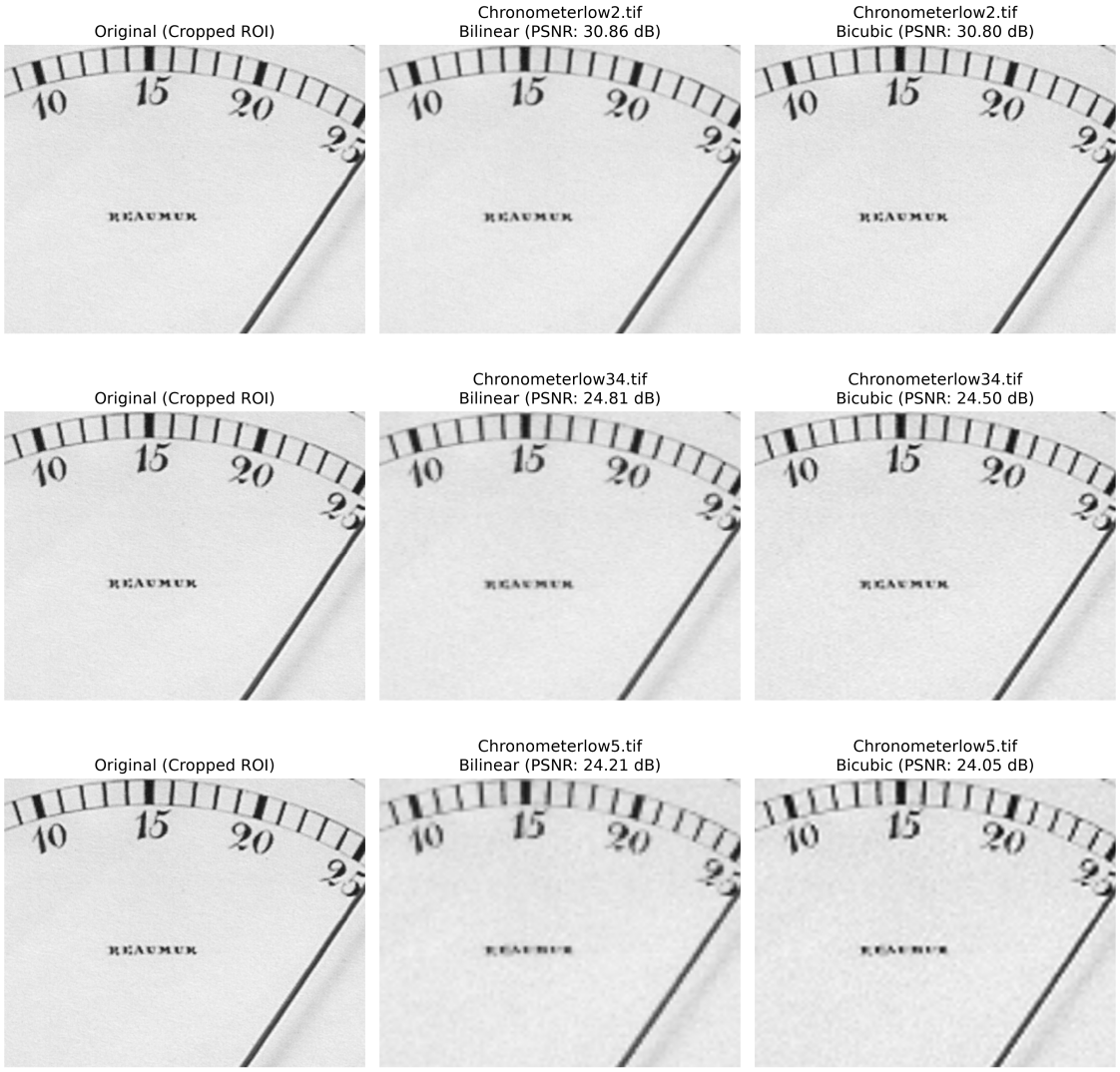


Figure 1: Visual comparison of cropped regions of interest (ROI) reconstructed via Bilinear and Bicubic interpolation from each sub-sampled input image against the original ground-truth image.

A Appendix: Source Code

```python
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import os

# ==========================================
# PART 1: SCALING (MIN-MAX NORMALIZATION)
# ==========================================
def scale_image(A: np.ndarray) -> np.ndarray:
    """Scales an input image matrix A to the range [0, 1]."""
    # Convert input matrix to float64 to prevent integer division truncation
    A_float = A.astype(np.float64)

    # Compute global minimum and maximum intensity values
    a_min = np.min(A_float)
    a_max = np.max(A_float)

    # Avoid division by zero for constant intensity images
    if a_max == a_min:
        return np.zeros_like(A_float)

    # Normalize intensities linearly to [0, 1]
    return (A_float - a_min) / (a_max - a_min)

# ==========================================
# PART 2: INTERPOLATION ROUTINES
# ==========================================
def bilinear_interpolation(img: np.ndarray, target_shape: tuple) -> np.ndarray:
    """Resizes a 2D image to target_shape using bilinear interpolation."""
    h_in, w_in = img.shape
    h_out, w_out = target_shape

    # Map output grid coordinates back to input image coordinates
    r = np.linspace(0, h_in - 1, h_out)
    c = np.linspace(0, w_in - 1, w_out)
    r_grid, c_grid = np.meshgrid(r, c, indexing='ij')

    # Identify surrounding integer pixel coordinates
    r0 = np.floor(r_grid).astype(int)
    c0 = np.floor(c_grid).astype(int)
    r1 = np.clip(r0 + 1, 0, h_in - 1)
    c1 = np.clip(c0 + 1, 0, w_in - 1)

    # Calculate fractional weights
    dr = r_grid - r0
    dc = c_grid - c0

    # Compute weighted sum of the 4 neighboring pixels
    out = ((1 - dr) * (1 - dc) * img[r0, c0] +
           (1 - dr) * dc       * img[r0, c1] +
           dr       * (1 - dc) * img[r1, c0] +
           dr       * dc       * img[r1, c1])
    return out

def cubic_kernel(t: np.ndarray, a: float = -0.5) -> np.ndarray:
    """Evaluates the Keys bicubic convolution kernel for distance t."""
    abs_t = np.abs(t)
    weights = np.zeros_like(abs_t)

    # Inner region: |t| <= 1
    mask1 = abs_t <= 1
    weights[mask1] = (a + 2) * (abs_t[mask1]**3) - (a + 3) * (abs_t[mask1]**2) + 1
```

```
    # Outer region: 1 < |t| < 2
    mask2 = (abs_t > 1) & (abs_t < 2)
    weights[mask2] = a * (abs_t[mask2]**3) - 5 * a * (abs_t[mask2]**2) + 8 * a * abs_t[mask2] - 4 * a
    return weights

def bicubic_interpolation(img: np.ndarray, target_shape: tuple) -> np.ndarray:
    """Resizes a 2D image to target_shape using separable bicubic interpolation."""
    h_out, w_out = target_shape

    def interpolate_1d_along_axis0(arr: np.ndarray, out_len: int) -> np.ndarray:
        in_len = arr.shape[0]
        coords = np.linspace(0, in_len - 1, out_len)
        x0 = np.floor(coords).astype(int)
        result = np.zeros((out_len, arr.shape[1]), dtype=np.float64)
        # Sum over the 4 neighboring points in 1D
        for m in [-1, 0, 1, 2]:
            idx = np.clip(x0 + m, 0, in_len - 1)
            w = cubic_kernel(coords - (x0 + m))[:, np.newaxis]
            result += arr[idx, :] * w
        return result

    # Separable execution: vertical pass followed by horizontal pass
    temp = interpolate_1d_along_axis0(img, h_out)
    out = interpolate_1d_along_axis0(temp.T, w_out).T
    return np.clip(out, np.min(img), np.max(img))

# ==========================================
# PART 3: PERFORMANCE METRICS & EVALUATION
# ==========================================
def compute_metrics(orig: np.ndarray, recon: np.ndarray) -> tuple:
    """Computes Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR)."""
    mse = np.mean((orig - recon) ** 2)
    psnr = 10 * np.log10(1.0 / mse) if mse > 0 else float('inf')
    return mse, psnr

if __name__ == "__main__":
    files = ["Chronometer.tif", "Chronometerlow2.tif", "Chronometerlow34.tif", "Chronometerlow5.tif"]
    images = {}
    for f in files:
        if os.path.exists(f):
            im = np.array(Image.open(f))
            if im.ndim > 2:
                im = im[:, :, 0]
            images[f] = scale_image(im)

    orig_img = images["Chronometer.tif"]
    target_res = orig_img.shape
    low_files = ["Chronometerlow2.tif", "Chronometerlow34.tif", "Chronometerlow5.tif"]

    for lf in low_files:
        rec_lin = bilinear_interpolation(images[lf], target_res)
        rec_cub = bicubic_interpolation(images[lf], target_res)
        mse_lin, psnr_lin = compute_metrics(orig_img, rec_lin)
        mse_cub, psnr_cub = compute_metrics(orig_img, rec_cub)
```

Listing 1: Python implementation of image scaling, bilinear/bicubic interpolation, and error metrics.